

Difference between Abstract Factory and Factory design pattern in Java



Let see another example of Abstract Factory and Factory design pattern in Java from JDK itself to get a better understanding. If you have done some XML work in Java e.g. [reading XML files using DOM parser](#), you may be familiar with `DocumentBuilderFactory` class which is an example abstract factory design pattern because it returns a factory called `DocumentBuilder` which then used to create `Document`.

```
//Example of Abstract Factory and Factory design pattern in Java  
DocumentBuilderFactory abstractFactory = DocumentBuilderFactory.newInstance();  
DocumentBuilder factory = abstractFactory.newDocumentBuilder();  
Document doc = factory.parse(stocks)
```

In this example `DocumentBuilderFactory` (Abstract Factory) creates `DocumentBuilder` (Factory) which creates Documents (Products).

Let's see some more difference between Abstract Factory and Factory design pattern in Java in point form :

1) One more *difference between Abstract Factory and Factory design pattern* is that `AbstractFactory` pattern uses composition to delegate responsibility of creating object to another class while Factory design pattern uses [inheritance](#) and relies on derived class or sub class to create [object](#).

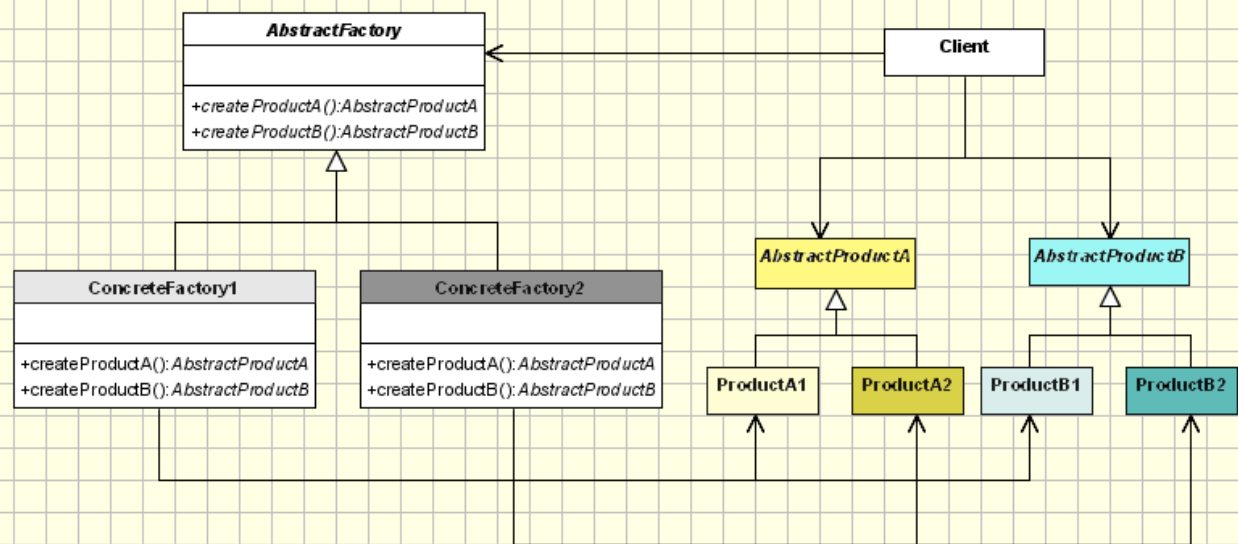
2) Abstract Factory may use Factory design pattern for creating objects but they are not just limited to that they can also use [Builder design pattern](#) to build object by doing series of steps or `Prototype` pattern to build object by copying or customizing prototype of that object. It completely depends upon your implementation whether to use Factory pattern or Builder pattern for creating products.

`javax.xml.validation.SchemaFactory` is another example of abstract factory.

Examples can found here:

<http://www.ibm.com/developerworks/xml/library/x-javaxxmlvalidapi/index.html>

cd: Abstract Factory Implementation - UML Class Diagram



```

abstract class AbstractProductA{
    public abstract void operationA1();
    public abstract void operationA2();
}

class ProductA1 extends AbstractProductA{
    ProductA1(String arg){
        System.out.println("Hello "+arg);
    } // Implement the code here
    public void operationA1() { };
    public void operationA2() { };
}

class ProductA2 extends AbstractProductA{
    ProductA2(String arg){
        System.out.println("Hello "+arg);
    } // Implement the code here
    public void operationA1() { };
    public void operationA2() { };
}

abstract class AbstractProductB{
    //public abstract void operationB1();
    //public abstract void operationB2();
}

class ProductB1 extends AbstractProductB{
    ProductB1(String arg){
        System.out.println("Hello "+arg);
    } // Implement the code here
}

class ProductB2 extends AbstractProductB{
    ProductB2(String arg){
        System.out.println("Hello "+arg);
    } // Implement the code here
}
    
```

```

abstract class AbstractFactory{
    abstract AbstractProductA createProductA();
    abstract AbstractProductB createProductB();
}

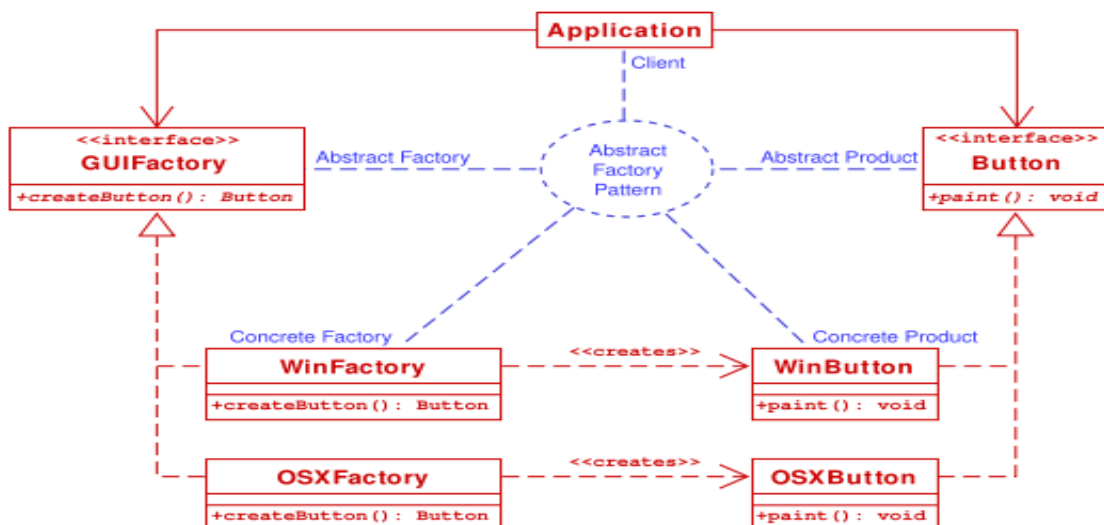
class ConcreteFactory1 extends AbstractFactory{
    AbstractProductA createProductA(){
        return new ProductA1("ProductA1");
    }
    AbstractProductB createProductB(){
        return new ProductB1("ProductB1");
    }
}

class ConcreteFactory2 extends AbstractFactory{
    AbstractProductA createProductA(){
        return new ProductA2("ProductA2");
    }
    AbstractProductB createProductB(){
        return new ProductB2("ProductB2");
    }
}

//Factory creator - an indirect way of instantiating the factories
class FactoryMaker{
    private static AbstractFactory pf=null;
    static AbstractFactory getFactory(String choice){
        if(choice.equals("a")){
            pf=new ConcreteFactory1();
        }else if(choice.equals("b")){
            pf=new ConcreteFactory2();
        } return pf;
    }
}

// Client
public class Client{
    public static void main(String args[]){
        AbstractFactory pf=FactoryMaker.getFactory("a");
        AbstractProductA product=pf.createProductA();
        //more function calls on product
    }
}
    
```

Class diagram



```

interface Button {
    void paint();
}

interface GUIFactory {
    Button createButton();
}

class WinFactory implements GUIFactory {
    public Button createButton() {
        return new WinButton();
    }
}

class OSXFactory implements GUIFactory {
    public Button createButton() {
        return new OSXButton();
    }
}

class WinButton implements Button {
    public void paint() {
        System.out.println("I'm a WinButton");
    }
}

class OSXButton implements Button {
    public void paint() {
        System.out.println("I'm an OSXButton");
    }
}

```

```

class Application {
    public Application(GUIFactory factory) {
        Button button = factory.createButton();
        button.paint();
    }
}

public class ApplicationRunner {
    public static void main(String[] args) {
        new Application(createOsSpecificFactory());
    }

    public static GUIFactory createOsSpecificFactory() {
        int sys = readFromConfigFile("OS_TYPE");
        if (sys == 0) return new WinFactory();
        else return new OSXFactory();
    }
}

```